MARVIN — Technical Reference

The exhaustive companion to the white paper: every subsystem, its logic, the decision record behind it, and pointers into the code. Written for contributors and deep evaluators. Covers v0.1.118 (2026-10-14). Where this document and the repository disagree, the repository wins.

> **Since v0.1.55.** Subsystems added after this document's original scope, each with its own decision record: the in-process **pty terminal** (ADR-0078), **background subagents** and built-in read-only agents (ADR-0080), **implementer subagents on isolated git worktrees** (ADR-0081 — the one amendment to the single-assistant rule), **Claude plan usage** from `rate_limit_event` (ADR-0082), the **escalating graph-drift rail** (ADR-0083), **blast-radius and pre-ship impact** queries (ADR-0084/0085), **dependency bootstrap and update checks** (ADR-0086), **Obsidian vault integration** and the plans canvas (ADR-0089 → 0092), the **advisor verdict parser** and its caveat record (ADR-0094/0095, amended 2026-08-31), and **provider-aware model resolution** for OpenRouter (ADR-0096). Two cross-cutting lessons are recorded as ADR-0097 (verify against the surface that *runs*, not the one that *reports*) and ADR-0098 (a rail keyed on vendor tool names is only as durable as those names). Since 2026-09-01: an **LSP client** whose diagnostics come from the buffer (ADR-0099), `/refine` proposing practice lessons (ADR-0101), **two sessions in one worktree** with the collision surfaced (ADR-0102), a **derived implementer-branch lifecycle** (ADR-0103), the **ship-review gate** at `git commit` (ADR-0104, §2.7) and the **practice loop** (ADR-0105, §2.7 and §4). Since 2026-09-07: **vertical-slice milestones** — a Phase 5 rule, a `relations` filter on `graph_path` (§3.3), and the `plan.horizontal` / `plan.vertical` kinds whose layers are discovered from the project's graph (§2.7, §4). v0.1.106, same day: the plan spine (`PlanModel.swift`) excludes a Definition-of-Done / Scope-of-Done / Acceptance-criteria block from step parsing and renders it verbatim, strips a leading checkbox from step ids so a plan re-seeded from its file keeps its progress, seeds `[x]` as completed, and heals an already-absorbed spine on the next reconcile (ADR-0052 / ADR-0068 lineage; 7 new tests, 641 assertions). Since 2026-09-10: **nothing survives a tab without a commit** — sweep, dry-run verdicts, Sync (ADR-0111, §3.x), **one place for each thing** in the multi-session chrome (ADR-0112, §8), the **backlog as the shared task list** — claims, live-store-only reads, resolution notices (ADR-0113, §3.x and §4), and the layout-pass hook of the constraint-loop breaker (ADR-0062 addendum 7). Since 2026-09-11: **one vocabulary for the left pane** — shared chrome whose decisions live in `MARVINLogic` so they can be asserted (ADR-0114, §8.2). v0.1.113–0.1.114: tab branches are never pushed or requested (ADR-0109 second amendment, §3.x), **full auto** as a third permission strategy (ADR-0115, §2.1), and a layout-oscillation probe (ADR-0062 addendum 8, §8.1). v0.1.115 (2026-10-01): **MCP trust by provenance** (ADR-0117, §3.3); **the context budget, second pass** — one system prompt per session, real settings isolation, indexes as a recent slice, a ~4.4K base prompt with MARVIN's own skills (ADR-0118, §3.4, §6, §11); **instruction files** read the way other agents write them (ADR-0119, §3.4); **stalls** — keep-awake, the stdin search deny, job completions never dropped (ADR-0120, §2.7, §9); **own-change consequences** — cross-layer impact, a commit gate, a scope-met check (ADR-0121, §2.7, §3.3); worktree state derived off the request thread, squash merges and orphaned checkouts reclaimed (§3.x); Agent SDK 0.3.280 (§10); and the transcript freeze and connection-pool starvation fixed (§8.1). v0.1.116 (2026-10-01): an orphaned sidecar no longer wedges on its dead stdout pipe; history paging cannot overflow or silently drop a page; the status-bar counters survive a relaunch (§8.1). v0.1.117 (2026-10-06): **a tab gets its worktree or is refused** (ADR-0122, §3.x); **parallel-tab guards** — numbered-file collisions, pattern kills, scope-met is not a decision (ADR-0123, §2.7); **shell reads through the graph gate**, a graph per worktree tab, a sidecar-held plan spine (ADR-0124, §1.4, §2.7, §3.2); large files open without a main-thread hang and the CLI's system notices render (§8.1); Agent SDK 0.3.286 (§10). v0.1.118 (2026-10-14): **a model for every helper role** (ADR-0126) — scout, graph-extractor, background model, commit messages, practice drafts, session auditor and skill discovery resolve through `resolveRoleModel` (`runtime/src/role-models.ts`) against the active provider's live catalogue, a stale pick falling back to the

> role's default; `GET` / `PUT /api/models/roles` and a Helpers section in the Models dialog. **Claude's server-side advisor** (ADR-0125) is a role too: off by default, `CLAUDE_CODE_DISABLE_ADVISOR_TOOL=1` when off, `settings.advisorModel` when on.

Paths are relative to the repo root. `runtime/` abbreviates `sidecar/packages/runtime/src/`. ADRs for **this repo** live at `docs/decisions/`; when MARVIN writes ADRs into a **user's project** they go to `<workDir>/docs/adr/`.

Vocabulary (Claude Agent SDK terms used throughout): `canUseTool` — the SDK callback invoked before every tool execution; MARVIN's gate lives there. `agentID` — present on tool calls made by a spawned subagent. `TodoWrite` — the SDK tool the model calls to maintain its task checklist. `Task {subagent_type}` — how a subagent is spawned. `disallowedTools` — SDK-level tool denial for an agent. Graphify terms: edges carry EXTRACTED / INFERRED / AMBIGUOUS confidence tags; "god nodes" are the highest-degree nodes (the de-facto architectural anchors).

---

1. Core loop & modes

1.1 The single-assistant loop

MARVIN is one Claude session in a continuous user↔assistant loop (ADR-0001). Workflow "roles" are phases the one assistant moves through, not agents that hand off to each other. This is the founding constraint everything else respects; the 2026 multi-agent literature it is based on reports up to 70% degradation on sequential code work and 17× error amplification in flat agent topologies.

1.2 The 8-phase workflow

Intake → Discovery → Impact analysis → Architecture → Plan → Implement → Verify → Ship, encoded in `runtime/personality.ts` (`CORE_BEHAVIOR`) and explained in `docs/concepts/eight-phase-workflow.md`. Phases are labelled in chat (`**[Phase N · Name]**`); mutating tools are forbidden before Phase 6; trivial changes take an explicit fast-path. Phase 6 carries a bounded self-remediation contract and Phase 7 a surface-and-offer contract (§11.3).

1.3 Modes: Ask / Agent / Plan

A `mode` axis orthogonal to the permission strategy (ADR-0036):

Mode	Mechanism	Effect
ask	`readOnly` flag in `classifyToolCall` + SDK `disallowedTools` backstop	every mutating tool hard-denied; reads and graph queries allowed
agent (default)	normal gate	full loop under the auto/gated strategy
plan	read-only planning turn on the **advisor/planner model**; plan presented inline; approval runs a **separate Agent turn on the executor model**	strategy approved before execution; re-planning doesn't arise because execution is not a planning turn

1.4 The plan spine

The plan subsystem is MARVIN's most-revised area; the current design:

- **Plan card** — a plan reply must open `# Plan — <title>`; the native renderer shows it as a collapsible card; detection is content-shaped so it survives transcript replay (ADR-0036 addendum).

- **Two tiers** — a bare `TodoWrite` checklist renders as a neutral **Task list**; an approved plan renders as a purple **Plan —** `<title>` strip that persists and ticks off in place (`macos/MARVIN/TodoListView.swift`).
- **Reconcile, don't clobber** (ADR-0046) — the active plan owns hierarchical `PlanStep`s; an incoming `TodoWrite` reconciles into them (matched step → status update, unmatched item → nested sub-task) instead of replacing the list. Completion is computed over top-level steps only. Plans live in a revision-aware session list.
- **Join keys + roll-up** (ADR-0049) — the executor tags every `TodoWrite` item `[N]` / `[N.M]`, giving a stable join immune to rewording; a step with sub-tasks completes **iff** every sub-task completes (hard invariant, v0.1.50).
- **Persistence** — a presented plan is auto-written to `<workDir>/.marvin/plans/<slug>.md` and opened in the editor; `PlanFile.render` overlays `[x]` on completed steps and appends discovered work, re-persisted on every reconcile. On session load, `replay` reconstructs plan + checklist from the transcript.
- **Continue anchoring** (ADR-0050) — the Continue control injects the active plan's concrete steps plus a guardrail: resume *only this plan*, no re-auditing.
- **Plan-in-context** (ADR-0051) — the client sends an authoritative `planContext` snapshot every turn; the runtime appends it as a `<system-reminder>` suffix on the user message (the volatile, uncached tail — prompt-cache-safe, never persisted), so compaction cannot lose the plan.

Code pointers for the spine: the plan model, reconcile logic, `PlanFile` rendering, and `replay` reconstruction live on the app side in `macos/MARVIN/ChatPreviewView.swift`; the two-tier strip and plan card are `macos/MARVIN/TodoListView.swift` and `PlanCardView.swift`; the per-turn `planContext` injection is in `runtime/sdk-runner.ts`.

1.5 Interactive decisions

When the model reaches a genuine fork it calls `AskUserQuestion`; the gate routes it through the confirm channel (§2.1) and the native `AskQuestionSheet` renders clickable options (single/multi + "Other"), returning the pick as the tool result (ADR-0040). The sheet registers with **no auto-deny timeout** — a human weighing a decision is never silently overridden (v0.1.40 fix).

1.6 Session history

Turn transcripts hydrate tail-first: the last 200 events paint instantly; "show 200 earlier / show full log" pages the rest in with an "N of M" count (ADR-0048). The 120MB worst case is reachable but never auto-loaded.

---

2. Permission & safety

2.1 The structural gate

`canUseTool` (`runtime/sdk-runner.ts`) runs **before** every tool execution. Both permission strategies flow through the same classifier, `classifyToolCall` — there is no second policy hidden in a closure (ADR-0015):

- **auto** — everything not hard-denied runs; every mutating call is still classified and written to a JSONL audit log (`runtime/auto-audit.ts`, read via `GET /api/audit`). Bypass-only allowances are tagged `auto-mode bypass:` so a later audit can see what gated mode would have prompted on.
- **gated** — three-way classification: auto-allow (reads, whitelisted commands), confirm (Edit/Write/unlisted Bash → a confirm card with the exact diff), hard-deny.
- **full** (ADR-0115) — auto, minus the three *containment* confirms (lane, shared-tree HEAD move, a worktree tab reaching the main checkout), each allowed call logged as `full-auto-mode bypass`. Metered CI, `AskUserQuestion` and plan approval still reach the user. The pill cycles gated → auto → full auto, `full` is

the only red control in the bar, and an unknown stored value parses to **gated**, never to the most permissive.
- **Hard-deny floor** — destructive shell patterns (`rm -rf /`, force-push to main, credential-file writes) short-circuit in **every** strategy.

2.2 The subagent read-only invariant

Any tool call carrying an SDK `agentID` that would mutate the workspace is hard-denied in `classifyToolCall` (ADR-0030). This single choke-point is what makes every subagent pattern (§5) safe; it cannot be configured off. Precision on "would mutate": denial of the file-editing tools (Write/Edit/NotebookEdit) is structural — the tool identity is unambiguous — while mutating *Bash* is the gate classifier's judgment (pattern lists + policy), a distinction worth keeping in mind when evaluating the guarantee. Contrast §2.7's design hooks, which *are* configurable; this invariant and the hard-deny floor are not.

2.3 Change checkpoints & review

The gate snapshots each file's pre-image the first time a session touches it (`runtime/change-checkpoints.ts`, disk-backed under `<dataDir>/checkpoints/`) — ADR-0034. The review window diffs against that snapshot (not git HEAD, which would show the user's own uncommitted work as MARVIN's). Accept advances the baseline; **reject reverse-applies the diff**, restoring the user's uncommitted state — something `git checkout` could not do. Committing clears reviewed-clean files (`reconcileCommitted`). Known v1 blind spot, documented in the ADR: Bash-driven mutations are not pre-imaged.

The review surface is a dedicated resizable window: side-by-side with Split/Inline toggle, single-column rendering for added/deleted files, virtualized rows, and >1500-line diffs gated behind "Show anyway".

2.4 The file-system write channel

User-initiated tree operations go through a second gated channel (ADR-0008): `fsWritePolicy` (`sidecar/packages/tools/src/fs-write-policy.ts`) classifies seven ops. Delete-to-Trash is auto (reversible); permanent-delete and secret-file writes (`.env*`, `*.pem`, `id_rsa*`) are confirm-danger; writes cap at 5MB. Confirm tokens are session-scoped, one-shot, 60s TTL. Both read and write routes share `fs-sandbox.ts`: canonicalization, symlink + ancestor-symlink escape rejection, path-length and NUL checks. OS→tree uploads require the `X-Marvin-Client` header plus size caps (ADR-0009).

2.5 The git channel

All git mutations flow through `sidecar/packages/git/` (ADR-0012): `runGit` is the only place that shells to git (`shell:false`, `GIT_TERMINAL_PROMPT=0`, timeouts, output caps); `argv-guards` whitelist every ref/pathspec/remote/message and reject flag-injection vectors (`-c`, `--exec-path`, `--upload-pack`, …) as a last line against RCE. `gitWritePolicy` classifies each op: `push --force` to main is always hard-denied; `--force-with-lease` is confirm-danger; dirty-tree branch switches and pulls are denied. Credentials are **inherited, never handled** (ADR-0013): MARVIN never writes child stdin, never transforms remote URLs, never prompts for git credentials; the user's own helpers (osxkeychain, gh, 1Password) answer. Remote stderr is classified into a stable error taxonomy for the UI.

2.6 Background-execution denial

Shell backgrounding (`cmd &`, `nohup`, `setsid`, `disown`) and the SDK's `run_in_background` are gate-denied (ADR-0038, ADR-0032) — a detached process would finish unreported. The honest alternative is §9.1.

2.7 Runtime design hooks

`runtime/design-hooks.ts` enforces the most load-bearing behavioral rules **structurally**, below the prompt. Four are built in: a blind source-file read on a codebase question is denied until the graph is consulted (graphify-first); a stale graph escalates a nudge (graph-drift, ADR-0083); an edit in security/schema/CI/migration paths is denied until an advisor consult has happened (advisor-on-ADR); and, since v0.1.102, a `git commit` is refused until the review skills the personality requires have run (`checkShipReview`, ADR-0104). The ship-review gate reads the diff the commit seals — the index plus whatever the same command stages: a boundary path (auth / creds / CI / sudoers / `.env` / `*.sh` / migrations) requires both `pr-review` and `security-audit`, more than three files or fifty lines requires `pr-review`, docs-only and lockfile-only pass; a review this turn covers the turn, an earlier one holds until the next commit; two denies per skill per turn, then allow and log; fails open on git errors. Since v0.1.115 the same `builtin:ship-review` row also covers `checkShipImpactRequired` (ADR-0121): a reviewable commit is refused until `graph_change_impact` has run this turn, and the refusal says to act on callers *and* cross-layer references. At a scope-met close, `ownChangeConsequences` (`workflow-guard.ts`) matches the provisional backlog items this session parked against the session's own changes — first-parent, non-merge commits plus uncommitted files in a worktree tab, files written in a shared one — and sends a match back as part of the change. The ADR-0057 guard now also recognises ADRs under `docs/adr/`.

Shell reads and the graph pointer (ADR-0124, v0.1.117). `classifyBashSourceAccess` (`bash-search.ts`) returns every source read a shell command performs — `cat` / `nl` / `less` / `bat`, `sed -n` ranges, `head` / `tail` counts, `awk` NR ranges, `grep` / `rg` / `ag` / `find` / `fd` — through `cd … &&`, loops and `xargs`, quote-aware. It feeds the existing rail: graphify-first, the drift tallies and the practice extractor all count a shell read as a source read. `builtin:graph-pointer` (`checkGraphPointer`) then refuses only the two wide shapes once the graph has answered this turn: a whole-file read of a source file over 200 lines, and a search rooted at the repository or at a directory no graph result pointed into. Pointers are lifted from every `mcp__marvin-graph__*` result by a PostToolUse hook. The **plan spine** gains a sidecar writer: `seedTurnSpine` turns a brief's checklist into a `[1]…[N]` pending plan before the model's first tool call, and `builtin:plan-spine` refuses a repeat commit — and the Stop hook a scope-met close — while seeded steps are open and no TodoWrite has run. **Parallel tabs** (ADR-0123): `checkNumberedCollision` refuses a commit that adds a numbered file (`0491-…`, `V2026…__…`) whose number a live sibling worktree or HEAD already holds, and suggests a free one; the tool policy refuses `pkill`, `killall` and `kill $(pgrep …)` in every mode — a tab kills only PIDs it started.

Stdin searches (ADR-0120). `checkSearchWithoutPath` runs first in `runDesignHooks`: `rg` / `grep` / `egrep` / `fgrep` with a pattern and no path operand, not fed by a pipe and with no `<` redirect, would read the SDK's never-closing stdin, and is denied with the fix in the message. The parser handles quotes, `--`, `-e` / `-f`, per-tool value flags (`-r` takes a value in rg, is recursive in grep) and `$( … )` inside double quotes, fails open on what it cannot parse, and carries the two-denies brake.

Practice rules (ADR-0105) are the fifth family and the first driven by data rather than code. `checkPracticeRules` reads the rules the user accepted in the Practice pane from `~/.marvin/practice/<projectId>/` (mtime-cached) and enforces each at its tier: `prompt` (a line in the system prompt, via `practicePromptBlock`), `nudge` (an advisory line on the tool result) or `deny` (a refusal, offered only for kinds with a machine-checkable discharge). Since v0.1.103 the four built-in gates are themselves rows in that store (`builtin:*`), so their tier, on/off state and message are editable from the pane; when no row exists the gate runs natively, which is why the 93 gate tests did not move. ADR-0104's brakes apply to every family: a deny needs a discharge path, `measure` mode logs instead of denying, two denies per rule per turn, and a bypass is logged — since v0.1.107 the graphify-first and advisor-on-ADR gates included (`BUILTIN_GATE_MAX_DENIES`; they had been one-shot), with the bypass counted on the gate's row. v0.1.107 also made `regressed` a rate (two recurring sessions at half the pre-acceptance rate), added an evidence threshold to the proposal test, moved the runner to `runPracticeAsync`, and shipped extractor v7 after an audit found five of six regressed rows on a real project to be the loop misreading its own gates (ADR-0105 addendum). Since v0.1.104 three more MARVIN-

owned hooks came out of the first practice report: a bare project-local skill name is rewritten to its plugin-namespaced form at the gate (the ADR-0058 `updatedInput` mechanism), a `PostToolUseFailure` hook remembers a failed Bash command for the turn so an identical re-run gets an advisory nudge, and the turn-close hook blocks, once, a three-plus-edit turn under an open plan that never updated `TodoWrite`. Since v0.1.105 the loop measures **plan shape**: `plan.horizontal` (a plan of three or more `[N]` milestones each confined to one area of the project, none crossing two) against `plan.vertical` (some milestone crosses, or is marked as a slice), with a `nudge`-tier template that fires on the `TodoWrite` presenting a horizontal plan via a `planShape` trigger. The areas are **discovered** by `discoverAreas` (`graphify-bridge/src/plan-areas.ts`): the project's code files, made relative to their common root, split recursively wherever a directory holds two or more substantial groups (≥ 3 files and ≥ 5 %, depth ≤ 3, dot-directories and test folders folded into their parent); each area's vocabulary is its own directory names and file stems, kept where ≥ 60 % of a token's occurrences are in that area and that share beats the area's share of all files by 0.2. A milestone crosses layers only on *structural* evidence — directory-level words from two areas, or a methodology marker (end-to-end, tracer bullet, thin slice) — and steps that open with verify / test / lint / ship / commit / docs / ADR count for nothing. No graph, or fewer than two areas, means no verdict. MARVIN ships no layer vocabulary. The same release fixed two extractors from the day's report at their source: a source read is now what the gate says it is (`bash-search.ts` holds the one classifier both use), and a review's report excludes the CLI-echoed skill body and stops at the commit (extractor v6).

Under the default `enforce` mode a deny is unconditional; `measure` logs what would have been denied; `off` disables the layer (`MARVIN_DESIGN_HOOKS`). These rules are belt-and-suspenders: prompt contract (§3.1, §5, §11) *plus* runtime hook — unlike the §2.2 invariant and the deny floor, which have no off switch. This layer exists because audits showed prompt-only rules decay as prompts grow: the 2026-09-02 session audit that produced the ship-review gate found the review skills invoked 0× across eight pushes of CI, sudoers and credential changes while every mechanical rule in the same session held.

---

3. Context & knowledge

3.1 Graphify-first

For structural questions the graph is queried **before** files are read — a one-line rule in `personality.ts`, the per-tool reference in the `marvin:graph-tools` skill, and a design hook that denies Read / Grep / Glob until a graph call has happened (`docs/concepts/graphify-integration.md`). Measured 27.5× cheaper per structural question (`graphify benchmark`, 2026-08-15; an earlier ~36× figure was an estimate, never measured) than file-reading; answers cite `file:line`.

3.2 Two graphs, three scopes

Per project (ADR-0028): a **code graph** (`graphify-out/graph.json`, AST-extracted) and a **knowledge graph** (`graphify-out/knowledge/graph.json`, heading structure + cross-links of docs/ADRs/memory/ `AGENTS.md`). The graph tools accept `scope: "code" | "knowledge" | "all"` (default `"code"`).

Every spawn goes through one resolver (`graphify-bridge/src/graphify-bin.ts`): `resolveGraphifyBin` searches `~/.local/bin` (uv, pipx), Homebrew, `/usr/local/bin`, `~/Library/Python/*/bin` and PATH and picks the **newest**; `GRAPHIFY_BIN` wins; a missing binary reports how to install it. The knowledge builder imports graphify as a library, so it runs under `graphifyPython()` — the interpreter named in the CLI's shebang — because the system `python3` cannot see a uv/pipx venv and exited 2 on every turn with stdio discarded (2026-09-29). The bundle ships `build-knowledge-graph.py` under `Resources/scripts/`; it had not, and every installed app's knowledge graphs had silently stopped refreshing.

3.3 The `marvin-graph` MCP

In-process server (`sidecar/packages/graphify-bridge/`), read-only, allowed at the gate on the SDK's provenance (below): `graph_summary`, `graph_search`, `graph_neighbors`, `graph_query`, `graph_path`, `graph_explain`, `graph_community`, `graph_god_nodes`, `graph_affected`, `graph_change_impact`, `graph_save_result` (with `outcome`), `graph_reflect`, plus diagnostics (`graph_diagnose`, `graph_benchmark`, `graph_index_schema`, `graph_export_callflow`).

Provenance (ADR-0117). Agent SDK 0.3.278 reports `mcpServer: { name, source }` to `canUseTool`, and `source: "sdk"` means an in-process server only the host can register. `mcpToolPolicy(name, provenance?)` keeps the allow for a MARVIN-prefixed tool unless the SDK reports another source, which then confirms (an unknown source is untrusted). Missing provenance keeps the name-based allow on purpose — subagent calls were not shown to carry it — and is logged as `gate.mcp_provenance_missing`.

`graph_affected` reads the directed `calls` edges of `graph.json`, oriented by call-site file (the source was the caller in 4,633 of 4,633 edges), with graphify's versioned AST cache as fallback. It had been answering from entries frozen on 2026-08-13: graphify moved the cache to `cache/ast/<version>/` and stopped caching JS/TS altogether.

`graph_change_impact` adds **cross-layer references** (`cross-layer-refs.ts`, ADR-0121) beside the call-graph callers: files outside the branch that name a changed config/script/infra file, a constant or config key the branch removed everywhere, or a path segment from a changed path literal, plus **parity gaps** — a value added among sibling constants that a spec outside the branch lists the siblings of but not it. One `git grep`; rules keyed on token shape, never on a stack; prose, generated output and MARVIN's own state skipped; labelled as text matching, not proof. Edges carry EXTRACTED / INFERRED / AMBIGUOUS confidence tags; god nodes and communities are first-class. `graph_path` (undirected BFS, in-process) takes a `relations` filter — `["calls", "imports", "imports_from", "method", "contains"]` for a path code actually takes rather than one threaded through a `references` string mention — and every hop carries the relation of the edge it leaves by and its `sourceFile`; the rendered arrows were off by one hop before 2026-09-07. This is Phase 5's tracer-bullet tool: entry symbol → persistence symbol is the first milestone's touchpoint list.

3.4 Lifecycle & the context budget

While the IDE has a project open, `/api/chat` fires debounced, fire-and-forget AST rebuilds of both graphs each turn — scoped to the active project, never MARVIN's own repo, zero LLM cost (ADR-0041). The same ADR budgets first-message context: `buildProjectContext` (`sidecar/packages/project-context/`) injects a graph header, the **ADR titles index** (not bodies), the **memory tail** (not the log), curated docs whole, and open backlog items. Measured effect: ~566K → ~13.4K tokens on a mature production project. The code-graph debounce is `refreshIntervalMs` = max(10 min, 10 × the last run), so a 65K-node graph that takes minutes cannot run most of the time while several tabs commit; background builds are reniced to 15, and a failed knowledge build warns once with its stderr tail.

The fixed load (ADR-0118). Project context was never the whole first request: a fresh tab on the same project sent **142,596 tokens** and died on "Autocompact is thrashing". Measured with the SDK's `getContextUsage` (logged every turn as `runagent.context_baseline`; `sidecar/scripts/context-baseline.ts` for a fresh tab), and fixed in four milestones:

1. **One system prompt per session.** `buildTurnSystemPrompt` always builds the full context and `turnSystemPrompt` sets `snapshot: true`, so the SDK records turn 1's prompt and replays it until compaction. Everything per-turn — orientation, mode, session tree, active plan — rides the `<system-reminder>` suffix (`turnReminders`). Before this, a resumed turn's different append *replaced* turn 1's, so from turn 2 the model had no project context from MARVIN, and every switch broke the prompt cache.

2. **Settings isolation that is real.** Omitting `settingSources` loads every source, so "isolation mode" had never been on: turns carried the user's Claude Code plugins (10 agents, hook-bearing plugins), the project `CLAUDE.md` twice and Claude Code's auto memory. Now `settingSources: ["user"]` (for `~/.claude/skills/`), every user-enabled Claude Code plugin off in the flag layer, and `CLAUDE_CODE_DISABLE_AUTO_MEMORY=1`.
3. **Indexes as a recent slice** — ADR titles, memory and backlog load as their newest entries plus the tool that fetches the rest (Golden Rule 5 amended; project documents still load whole). `marvin-memory`, `-backlog` and `-obsidian` are deferred behind `ToolSearch`.
4. **The base prompt** from 29.4K to ~4.4K tokens (§11).

Result on that project: **72,284 tokens**; a turn that starts over half its window shows a row saying so.

Instruction files (ADR-0119). `resolveInstructionFiles` reads `CLAUDE.md` / `.claude/CLAUDE.md`, and `AGENTS.md` when there is no `CLAUDE.md` or it carries distinct content — a near-copy (≥ 50 % of its lines in `CLAUDE.md`; real copies measure 85–88 %, a distinct file 9 %) is skipped and reported, pointing at the `marvin:instruction-files` skill. `@path` imports expand as Claude Code's do, confined to the project by real path. A `PostToolUse` hook surfaces a subdirectory's instruction files once per session when a Read/Edit/Write first reaches it, reset on compaction, never under dependency, build, VCS or `.marvin` directories.

3.5 Project fingerprint & infra probes

`fingerprint.ts` derives ~10–42 namespaced tags (`framework:next@16`, `architecture:multi-tenant`, `compliance:gdpr`, `test:playwright`) from manifests and code tells, cached at `.marvin/fingerprint.json` (ADR-0024); it drives skill enablement and recommendations (§6). Project-agnostic infra probes (`probeHttp`, `probeDockerContainer`) back Phase 2's "probe running infra" without any hardcoded service list. Both files live in `sidecar/packages/project-context/src/`.

3.x Per-session posture, and what a worktree does not isolate

Since v0.1.110 the seven per-turn controls — autonomy mode, executor and advisor model, executor and advisor effort, personality, permission strategy — are **per session** rather than app-wide. They live in a capped MRU store keyed by marvin session id; the former app-wide values became the defaults a new tab inherits. The sidecar had already modelled this correctly (`SessionPosture` in `session-meta.ts`, written on every turn start so a server-initiated resume can rebuild the turn), so the client hydrates a restored tab from the session-watch snapshot it already fetches. A local record always outranks the server's: the server's is history, the local one is a choice.

`MarvinBridge`'s posture fields were redefined to mean "the tab on screen", which made every existing reader session-aware without change. `setActiveMarvinSession` applies the arriving session's posture, in the bridge rather than at each call site, so a tab switch cannot come apart from a posture switch.

Isolation boundary. A session worktree isolates the working tree and the branch, and nothing else: the Docker daemon, host ports, databases, global caches and anything keyed on `$HOME` remain machine-global. `.marvin/worktree.json` carries an `env` map applied to every turn in a worktree-mode tab (and thus to every subprocess a turn spawns), validated at the boundary — POSIX names, 64 entries, 4 KB per value, malformed entries dropped. It is deliberately never auto-detected, unlike `symlinkDirectories`: which variables make a project's runs independent is not derivable from a repository, and guessing it would be project knowledge MARVIN must not hold (ADR-0110).

Integration cost. Branch integration is a local merge and never a push; `mergeAllWorktrees` folds every `ready` branch in one pass, oldest first, stopping at the first conflict. Commands that spend CI minutes — opening a merge/pull request, merging one, starting a pipeline — are classified by `classifyMeteredCiRisk` and confirmed

in **every** permission mode, alongside `AskUserQuestion`, plan approval and the shared-tree gate. `git push` is not classified: on a pipeline-gated project a push to a branch with no open request starts nothing (ADR-0109).

Tab lifecycle (ADR-0111). The close decision is derived from sidecar state (`SessionWatchWorktree`: `commits`, `dirty`, `behind`, `mergedInto`, `target`) rather than from what the client remembers. A closed tab with no commits and a clean tree, or a branch already contained in another ref, is removed without asking; a dirty tree is committed as work in progress on *Keep*; *Merge* names the main checkout's branch. `sweepWorktrees` runs at boot and after every close, skipping any tree whose session has a live turn. `previewIntegration` runs `git merge-tree --write-tree --name-only` against the target — nothing checked out, nothing moved — and the Sessions pane's *Ready to integrate* rows carry its verdict. **Sync** (`POST /api/sessions/sync`) resumes the owning session with a merge prompt so conflicts are resolved by the session that made the changes. A branch is renamed from its first commit's subject on the first turn that leaves one — read only from the tab's own first-parent, non-merge commits not already on its base, so a catch-up merge cannot rename it after a sibling's commit; an integrated tab that receives a message is recut from HEAD.

A worktree or a refusal (v0.1.117, ADR-0122). A full disk once made `git worktree add` fail and two new tabs ran silently in the shared checkout. There is no fallback any more: once a tab wants a worktree, any failure answers `409 worktree-failed` before a meta or transcript line is written. `addWorktreeAtomically` wraps a free-space floor (10 GB; `MARVIN_WORKTREE_MIN_FREE_GB` beats `.marvin/worktree.json`'s `minFreeGb`), `git worktree add`, an optional non-cone sparse checkout (`sparseExclude`) and the registry write, and rolls every step back on failure. Both keys are project-controlled input, validated where read. **Each tab queries its own graph** (ADR-0124): `resolveGraphRoot` picks `<worktree>/graphify-out/` when it exists, else the project graph with a note that the tab's changes are not indexed yet; `maybeRefreshWorktreeGraph` seeds it from the project graph and refreshes it AST-only at turn start and end, one graphify per project at a time.

Derivation off the request thread (v0.1.115). The reconcile reads git dozens of times per call — ~150–200 spawns on a project with 24 worktrees — and ran with `execFileSync` on request paths; a CPU profile put 4.9 s of every 25 s inside `/api/sessions/watch`, and every other request queued behind it. The derivation is now written once, as generators that *yield* each git read, and driven by `runSync` (mutation paths, tests) or `runAsync` (`reconcileWorktreesAsync`, `sweepWorktreesAsync` — every read path), so the two can never disagree about a state; a test asserts they derive identically. `session-watch.ts` serves each git fact stale-while-revalidate (5 s; callers share one in-flight refresh), and `?fresh=1` — sent by the tab close, which may discard a tree it believes empty — derives now. The close and boot sweeps run in the background under the busy-session guard; `previewIntegration` and `dryRunMerge` are async.

Merged, by content. `mergedIntoSteps` checks ancestry first, then whether merging the branch into its base or the main checkout's branch would change nothing (`git merge-tree` yields the target's own tree) — which is how a squash, rebase or cherry-pick merge is recognised; such branches had read `ready` forever (2.2 GB on one project). A partial pick or a conflict fails the test. An open tab whose commits are already integrated carries `mergedInto` while its state stays `session`, and the close treats it as merged. `adoptOrphans` also registers a checkout under `.marvin/worktrees/` on a branch MARVIN did not name (481 MB invisible to every surface before).

Bookkeeping never blocks a merge. The runtime writes advisor caveats, backlog, memory and security notes to the project root for every tab, and those files are tracked, so each branch carries a copy. `mergeWorktree` merges `.marvin/` bookkeeping with git's union driver through a per-invocation attributes file (the repo's own `.gitattributes` still wins), and sets aside uncommitted bookkeeping the branch also changed, restoring it unioned after the merge.

Tab branches stay local (ADR-0109 second amendment, v0.1.113). Pushing a `marvin/tab/*` branch or opening a request for it is **denied** in every mode, naming *Merge all* / *Prepare MR*; two tabs had each opened a merge request after the metered-CI confirm was allowed twice.

Shared backlog (ADR-0113). Items carry `claimedBy` / `claimedBranch` / `claimedAt` while **doing**; `backlog_claim` (MCP) and `PATCH /api/backlog {status:"doing", sessionId, branch}` take the claim, a second claimant gets the holder back (`409 held` over HTTP), `backlog_resolve` releases it. The near-duplicate refusal names the holder. `backlogSnapshotTarget` in the session-worktree gate denies `Read` / `Grep` / `Glob` / `Bash` aimed at `<worktree>/.marvin/backlog` in every permission mode, pointing at `backlog_list`; the root's copy is the live store. `notifyTabsOfResolution` composes one notice per resolution and delivers it to every other live tab of the project whose branch (`git diff --name-only base..branch`) touches a path the item names — injected into a running turn (ADR-0076) or queued for the next (ADR-0069), capped at ten, marked as coming from another tab.

4. Cross-session persistence

Content class determines the store; each store has an **enforced write path** that rejects the wrong class:

Class	Store	Write path	ADR
Durable facts (invariants, gotchas, constraints)	`.marvin/memory/<slug>.md` + one-line index	`remember` MCP tool (caps, supersede-by-name, rejects activity/status)	0042
Deferred actionable work	`.marvin/backlog/<slug>.md` + index	`backlog_add` / `backlog_claim` / `backlog_resolve` (consent-gated; `provisional:true` auto-parks at discovery; a claim names the holding tab)	0044, 0047, 0113
Material decisions	`<workDir>/docs/adr/NNNN-*.md` (user projects; this repo uses `docs/decisions/`)	Phase 4, enforced template with Scope-of-Done	triggers in §11.4
Status / activity	git history, changelog	ordinary commits	—
Session notes	`.marvin/session-notes.md`	the native Scope-met chip	0042
Behaviour — repeat failures, the rules about them, whether they held	`~/.marvin/practice/<projectId>/` (findings ledger, rules, runs; config and score weights shared across projects)	the Practice pane only, over `/api/practice`: approve / dismiss / fixed / escalate / adopt. The nightly runner writes findings, never rules	0105

On the backlog's two-step capture: `provisional:true` auto-parks a *memo* the instant something is noticed (no go-ahead needed — a memo is not a commitment); the consent gate is the keep/dismiss review at the scope-met handoff. Capture is automatic, retention is consented.

The practice store is the one layer under the data dir rather than the project's `.marvin/`, because it is about MARVIN, not the project. A finding is keyed by a fingerprint (`kind[:qualifier]`) that a deterministic extractor computes over a transcript, counted by *distinct session*, with day-two semantics (new / recurring / proposed / regressed / confirmed / dismissed-then-resurfaced) and a versioned extractor (v6 at v0.1.105). Twenty-two kinds ship, eighteen of them in nine failure/success pairs so a finding carries a rate; three are report-only (cache re-creation, repeated errors, over-budget turns) because they are about MARVIN's implementation, not a behaviour a rule can change. Scoring is a small linear model over recurrence, cost, rate, reliability and actionability, minus decay; the weights can be fit from every ledger's own outcomes (Spearman rank correlation, coordinate descent; cost-share ranking below eight labels) and are applied only on a click, with provenance. A project with fewer than three sessions cannot propose; it shows a calibration counter and offers rules confirmed in the user's other projects for adoption, scoped to this project with its own verification clock. Subagent calls are never counted. No model reads a transcript; the one model call in the pane drafts a rule's wording from aggregates, in a read-only two-turn session.

The design was forced by a real failure: a memory file activity-logged to 419KB (~~99% redundant with ADRs/git~~) ~~that overflowed the context window.~~ ~~`/memory-compact`~~ ~~migrates legacy logs. The backlog is a **parking lot, never a queue** — no agent pulls from it (Golden Rule 1); open items surface in the next session's context and in the macOS backlog panel. Transcripts (~~/.marvin/sessions//*.jsonl`, every event verbatim) and the cost ledger (~/.marvin/cost-tracker.json`, daily/weekly/lifetime per project) complete the picture.

---

5. Subagents

Three sanctioned patterns, all read-only by the §2.2 invariant, all bounded; anything else is forbidden by ADR-0001:

- **Advisor** (ADR-0007, ADR-0033) — a registered agent definition carrying the user's advisor model and its own reasoning effort; spawned via `Task {subagent_type:"advisor"}`. Blunt-critique structure: risks / alternatives / pushback / verdict. Eight MUST triggers (ADR writes, security-sensitive work, blast radius ≥5, non-backward-compatible changes, architecture tie-breaks, concurrency, crypto, user request) plus anti-triggers.
- **Scout** (ADR-0014) — breadth-first read-only research (`disallowedTools` at the SDK level + the gate invariant); MUST for 3+ parallel searches or context relief; MUST NOT be sequential-implementation or user-facing authority.
- **Dynamic workflows** (ADR-0030) — script-orchestrated parallel read-only fan-outs for audits / surveys / migration discovery; opt-in (high effort or explicit ask), never automatic, never implementation.

A standing pattern rather than a type: after drafting an ADR, an advisor "future-MARVIN critique pass" lists every question the ADR leaves unanswered; gaps are closed before the user sees it.

---

6. Skills

- **Installed ≠ active** (ADR-0037) — `skill-enablement.ts` computes the active set: explicit user choice, else core skills ∪ fingerprint-matched domain skills. The prompt names the active set each turn and tells the model to ignore the rest (measured 20 → 7 relevant on MARVIN's own repo). Per-skill toggles in the Skills pane persist to `.marvin/skills.json`.
- **Triggers** — each core skill (test-driven-development, systematic-debugging, pr-review, security-audit, frontend-design, graphify) has one trigger line in `personality.ts`. The enumerated MUST / MUST-NOT lists that followed the 2026-05-22 audit (five of six skills had fired ~0× across thousands of qualifying turns) were retired in v0.1.115 (ADR-0118): current models over-fire on emphatic lists, and the two that matter most at commit — `pr-review` and `security-audit` — are enforced by the ship-review gate (§2.7), which is the only deterministic layer.
- **MARVIN's own skills** — `marvin:adr`, `marvin:graph-tools`, `marvin:browser`, `marvin:skill-audit`, `marvin:workflow-audit`, `marvin:greenfield`, `marvin:instruction-files`. Their text lives in `runtime/core-skills.ts`; every turn writes them to `<MARVIN_DATA_DIR>/core-skills/` as a local plugin named `marvin`, so they always match the running version. A test checks every `marvin:` pointer in the prompt resolves and every core skill is pointed at.
- **Skill audit** (ADR-0024, ADR-0025) — a `## Skill audit pending` block injects until `.marvin/skills.md` exists; MARVIN owes exactly one chip-strip recommendation with two verbs never mixed: **install** (user-global, `~/.claude/skills/`) for language/framework/test tags, **build** (project-local, `.marvin/skills/`, PR-reviewable, via skill-creator's eval loop) for architecture/domain/compliance tags.
- **Fetch from Git** (ADR-0039) — "Add from GitHub" shallow-clones a repo / sub-path / plugin marketplace, discovers `SKILL.md` folders, installs; clone-and-copy only, never executes repository code.

---

7. Models & auth

- **Role routing** (ADR-0002, ADR-0003) — independent executor and advisor/planner model slots with per-role reasoning effort (ADR-0033); a Sonnet executor + Opus advisor combination cuts inference cost substantially on routine work (~30–40% by per-token pricing arithmetic; workload-dependent, not a benchmark). Plan mode runs on the planner slot, execution on the executor (§1.3).
- **Auth resolution** (`runtime/auth.ts`, `auth-config.ts`) — the supported credential is an **API key**: the key configured in Settings (a `0600` file at `~/.marvin/auth-config.json`, provider `anthropic` or `openrouter`) wins; otherwise env `ANTHROPIC_API_KEY`. Raw keys are never logged and never returned by any status surface (last-4 hint only). OpenRouter is reached by pointing the CLI at a local proxy (`/api/proxy/openrouter`) that forwards Anthropic-format requests, and every model id is then OpenRouter's (ADR-0096).
- **Not a supported credential: a Claude.ai subscription login.** Anthropic's authentication policy (Legal and compliance, February 2026) reserves OAuth sign-in for Claude Code and Anthropic's own apps and requires API keys for products built on the Agent SDK. The older host-credentials detection and the Keychain read for model discovery (ADR-0029) predate that policy; they are legacy, undocumented here, and not supported.

---

8. UI surfaces (macOS SwiftUI app)

The app (`macos/MARVIN/`) is fully native — the Tauri/WebView era was migrated out ship-of-Theseus style (ADR-0010/0011/0016 record the history). Major surfaces:

- **IDE shell** — three panes (file tree / chat / brain-graph), split-view autosave, light-first OKLCH theme with dark mode (ADR-0006).
- **Chat** — mode + effort controls in the input (`ChatModeToolbar`), per-project persisted tabs, image paste + attachments, plan cards, the two-tier checklist strip, decision sheets, compaction banner (ADR-0022), scope-met chip strip.
- **Sessions** (ADR-0107, 0111, 0112) — the tab strip is the only switcher (tabs compress to fit, ‹ › steppers, an overflow list, ordinals for duplicate titles), History holds closed sessions with ⇧⌘[/ ⇧⌘], the context header reads project then `own branch · name` with a facts-plus-actions popover, posture is one pill, a tool confirm is a card in the tray, and the Sessions pane has five sections — Needs you, Working, Ready to integrate (Merge / Merge all / Sync / Prepare MR), Idle, Recent — with inline Allow / Deny and ↑↓↵ navigation.
- **Editor** — STTextView with tree-sitter highlighting (Swift, TS/TSX, JS/JSX, Go, Rust, JSON), diff gutter positioned from real layout fragments, ⌘S with mtime compare-and-swap.
- **Navigation** — Quick Open (⌘P), graph-backed Go-to-Symbol (⌘T), ripgrep Find-in-Files with replace-all, file history popover, Quick Look.
- **Terminal** — PTY-backed with ANSI parsing; build-task palette (⌘⇧B) discovering tasks from package.json / Makefile / Package.swift / Cargo.toml; diagnostics panel parsing compiler output.
- **Source control** — stage/commit/push/pull/fetch through the guarded git channel (§2.5), branch line, remote-error banner with the stderr taxonomy.
- **Review window** — §2.3.
- **The seven left panes** — Files, Search, Source Control, Sessions, Skills, Plugins, Practice; one control vocabulary across all of them (§8.2), with Search jumping to the matched line, Replace All confirming before it writes, and every outcome carrying its kind.
- **Backlog panel** — browse / Done / Dismiss / Promote-to-plan (which switches to Plan mode and queues if a turn is live — v0.1.53). Rows are lazy, a status change updates the row in place and rolls back on failure (it used to refetch all 1,076 items, 1.6 MB, per click), and the first load shows a static skeleton rather than

"No open backlog items". Source Control and the transcript use the same `SkeletonRows`.

- **Context panel** — the status-bar `ctx` chip opens a live breakdown: exact resident/window % from SDK usage plus per-category estimates (system prompt · tools/MCP · project context · transcript · free).
- **The brain** (ADR-0019) — a Metal-rendered live state indicator with five behavioral states (idle / thinking / tool / writing / error).
- **Status bar** — health hairline, cost pill, branch badge, model row, graph-ops ratio.

8.1 Health & resilience

`HealthMonitor` demotes to `.offline` only after **3 consecutive** `/api/health` misses (5s timeout, fast re-poll while misses pend) — one slow poll from a busy sidecar no longer tears down and rebuilds the IDE (v0.1.54). One live turn per session is enforced server-side (`POST /api/chat` → 409 rather than evicting; Stop is authoritative via `cancelLiveTurn`); closing the window doesn't kill a running turn (resume via `GET /api/chat/resume`).

Three v0.1.114–115 fixes, each settled by a measurement after reading the code had produced wrong answers. **The transcript freeze**: the `LayoutOscillationProbe` (ADR-0062 addendum 8 — labels on transcript rows and the Sessions list via `onGeometryChange`) named assistant rows flipping between 45 pt and 69–76 pt 40 times a second; `RichText.sizeThatFits` answered a zero or non-finite width proposal with a made-up 400 pt, so the bottom-anchored lazy stack never converged. **One chat model**: `ChatPreviewModel.shared` replaces view-owned `@State`, because each split-view rebuild created a fresh model (`heap`: 68 alive after 64 rebuilds, each holding its announce stream open). **Two connection pools**: SSE streams (turn, resume, announce, live feed) use their own `URLSession` (16 per host), because six open streams filled the shared pool and a tab switch's transcript GET queued forever. A loading skeleton is an overlay on the transcript's `ScrollView`, never a row of its lazy stack — as a row it spun the main thread at 100 % in `LazyLayoutViewCache.updatePrefetchPhases`.

Three more in v0.1.116, from one crash report and one CPU profile. **The orphaned sidecar**: the sidecar outlives the app by design, but its stdout and stderr are pipes to it. Once the app died, every log write raised `EPIPE` as an uncaught exception, which Next's handler logged to the same dead pipe; the profile showed ~98 % CPU in `patch-error-inspect` source-mapping one stack, forever, and `/api/health` timing out — a relaunched app would have adopted a dead server. `guardStdio` (`stdio-guard.ts`) absorbs write errors on both streams at boot. **History paging**: "Show full log" sets the render window to `Int.max`; "Show 200 earlier lines" then added 200 and trapped on overflow. Widening goes through `HistoryWindow.widened`, which saturates; a page asked for during a live turn loads when the turn ends instead of being discarded, and a failed fetch says so in the paging row. **Counters after a relaunch**: a rebuilt tab replays only its last 200 lines, so `graph N · reads M` counted 3 of a session's 14 graph calls and hid under its 5-call threshold, and `ctx` came only from live events. `GET /api/sessions/[id]` now returns `toolCounts` for the whole transcript — a substring count over the raw JSONL, identical to a full parse on four real transcripts — and the replay restores `ctx` from the last usage it saw.

Two in v0.1.117. **Large files**: a 16 s hang report put every main-thread sample in `FileViewerNSView.applyHighlights`, which called `STTextView.addAttributes` once per syntax span — an editing transaction per span, each re-syncing the layout managers and copying a paragraph to recompute typing attributes. Spans, swatches and squiggles now go to the text storage in one transaction with one layout pass. **System notices**: Agent SDK 0.3.283+ delivers the CLI's warnings as `system/informational` and plugin load failures as `system/init.plugin_errors`; the reducer dropped every system event, so `SystemNotice` (`MARVINLogic`) turns warnings, notices and plugin errors into a quiet row, once per distinct text.

8.2 One vocabulary for the seven panes

Measured across Files, Search, Source Control, Sessions, Skills, Plugins and Practice before any of them was touched: **five** button styles, **five** private section headers, **four** corner radii, **three** registers of empty state, and a `PaneNotice` type with kinds and a glyph that exactly one pane used. The same concept was drawn a different

way in every pane, which is Apple's *Familiarity* principle failing at the level of the whole sidebar (ADR-0114).

The split is forced by the test target rather than chosen: `MARVINTests` links `MARVINLogic` and nothing else, so a rule inside a SwiftUI view cannot be asserted. Every **decision** is therefore a value or a function over values in `MARVINLogic/PaneChrome.swift` — `PaneEmptyState` with its factories, `PaneBadge.text` (clamped at `999+`), `PaneHeaderOverflow.plan(actionCount:)`, `PaneNotice.Dismissal` and `PaneNotice.kind(forOutcome:)` — and `MARVIN/Pane*.swift` draws them and holds no rules of its own.

Two of those decisions are load-bearing. `dismissal` **is decided by kind**, so success and info may leave on their own while a warning or error waits to be dismissed: three panes previously routed every outcome through one tinted strip on a four-second timer, which meant the message that mattered was the one certain to vanish unread. **Header overflow is decided by action count**, never by measuring the container — a `GeometryReader` that re-enters layout in order to decide layout is the oscillation ADR-0062 crashes on, and a rule a person can predict beats one that shifts under them as they drag the splitter.

Admission to the vocabulary is one rule: **three or more panes need it, and the signature carries no pane-specific parameter.** Row *content* fails it — a skill row carries a toggle, a version and a contribution list; a git file row carries a status letter and a path — so row *shape* ships as a modifier over a caller-built `HStack`, never as a container with nine parameters.

Three constraints bind every shared component, each from a measured regression. No component declares an intrinsic `minWidth`, and every pane root ends `.frame(minWidth: 0, maxWidth: .infinity).clipped()`: the seven panes stay mounted in one `ZStack` (`PaneSlot.keptMounted`), so the widest intrinsic minimum among them becomes the sidebar's, and the overflow pushes the 44pt activity rail off the window. No component is applied to a pane root and none makes a container `.focusable()`, which draws macOS's focus ring around an entire pane. And only `hovering`, `pressed` and `notice` are animated, never layout — a height transition inside a mounted, zero-framed slot re-lays out the whole pane.

---

9. Background & async

- **Background jobs** (ADR-0038) — `run_background_job({command, reason})` — the tracked replacement for the raw backgrounding mechanisms §2.6 denies (similar name, opposite fate: `run_in_background` is denied, `run_background_job` is the sanctioned tool) — spawns a tracked child, streams an 8KB output tail, and **fires a real follow-up turn on process exit** with the exit code and tail. Limits: ≤3 concurrent per session, chain depth ≤8. Shutdown signals (SIGTERM on app quit) are "stopped, not finished" — no spurious failure turns on relaunch (v0.1.48). Running jobs are kept in an on-disk ledger that boot reconciles, so a sidecar restart or crash reports "result unknown" instead of silence; a SIGKILL/SIGTERM from outside while MARVIN keeps running fires a "stopped, not finished" turn after a 5 s grace; background jobs inherit the worktree's validated `env` map.
- **Keep-awake** (ADR-0120) — `keep-awake.ts`, wired into `turn-registry.ts`: the first live turn spawns `caffeinate -i -w <sidecar pid>`, the last one's end kills it — one assertion however many tabs run, none outside a turn; darwin only, `MARVIN_KEEP_AWAKE=0` disables. Measured cause: 51 of 51 silent gaps in a 20-hour session began at a macOS `Sleep` and ended at a `DarkWake`.
- **Wakeups** (ADR-0031) — `schedule_wakeup` re-invokes MARVIN at a chosen time via a persistent, boot-re-armed scheduler (delay 60s–24h, ≤5 pending, re-schedule depth ≤3); a fired wakeup **yields** to a live interactive turn and re-arms. A plain wakeup gives up after 60 deferrals; a `background job done:` wakeup keeps yielding for six hours (`MAX_JOB_DONE_DEFERRALS`) — six job results had been dropped in one day by the 20-minute rule.
- **Announce SSE** (ADR-0043) — an always-on per-project stream re-attaches an idle client to any server-initiated turn (job completions, wakeups), with a "background job running" chip.

- **git-watch** (`sidecar/packages/git-watch/`) — per-workDir commit detection feeding graph auto-rebuild and inline commit surfacing.
- **The anti-fabrication contract** (§11.5) makes these three mechanisms the *only* honest ways to promise future work.

---

10. Distribution & operations

- **Homebrew** (ADR-0023) — `brew tap RobertIlisei/marvin && brew install --cask marvin-ai` (token disambiguated from the unrelated "marvin" cask). The .app bundles the Next.js standalone sidecar + a pinned Node 22 runtime; the SwiftUI process spawns and reaps it, reclaiming port 3030 first (ADR-0035) so "new app on disk, old code in memory" cannot recur.
- **Signing** (ADR-0026) — every release zip carries a minisign (EdDSA) signature; the public key is pinned in the app repo, the tap README, and the cask constant — three copies across two repos, so tampering with one is visibly inconsistent. Cask auto-verify is the ADR's Phase 2.
- **macOS 26 Gatekeeper** (ADR-0027) — ad-hoc-signed bundles are kernel-killed in `/Applications`; MARVIN installs to `~/Applications`, the cask strips quarantine in a `postflight`, and first launch needs the one-time "Open Anyway".
- **Lifecycle** — `bin/marvin start/stop/restart/status/logs/doctor/ knowledge-graph`; `install-macos-app --bundled` builds, bundles, health-probes on a probe port, then installs; `brew uninstall --zap` wipes `~/.marvin`.
- **Observability** — `/api/health` reports auth mode + version; the sidecar logs to `~/Library/Logs/MARVIN/sidecar.log`. Optional **Honeycomb OTEL export** (`runtime/honeycomb-telemetry.ts`): per-turn env injection so the SDK-spawned CLI emits spans to the *user's own* Honeycomb account — off unless the user configures it; keys redacted from all logs.
- **Agent SDK contract** (ADR-0073) — `@anthropic-ai/claude-agent-sdk` **0.3.286**. Pins in `sdk-runner.ts`: `TodoWrite` opted back in (`CLAUDE_CODE_ENABLE_TODO_TOOLS=1`, `CLAUDE_CODE_ENABLE_TASKS=0`) or the plan spine freezes; `alwaysLoad` on `marvin-graph` and `marvin-control` only; `snapshot: true` with one prompt per session; `settingSources: ["user"]`. `0.3.N` bundles CLI `2.1.N`, and the API rejects a CLI older than the model requires — the 0.3.280 bump was forced when every request, auto-compaction included, failed with "*Claude Code 2.1.278 does not support this model*", leaving an over-full session unable to shrink itself.
- **CI/release** — tag push → `release.yml` builds the .app on a macOS runner, stamps the version into Info.plist, zips, signs, publishes; the cask is then bumped with the published sha256.

---

11. Behavioral contracts (`personality.ts`)

The prompt is a contract surface, not vibes. Since v0.1.115 (ADR-0118 Milestone 3) it is ~4.4K tokens, down from 29.4K (91 MUSTs, 43 MUST NOTs, 54 ADR citations; `personality.ts` 2,070 → ~210 lines): one plain rule per concern, occasional procedures moved into the `marvin:*` skills (§6), memory and backlog content rules in the tool descriptions that enforce them, and a gate wherever a rule must hold regardless. `personality-surfaces.test.ts` guards the parsed formats verbatim, the moved rules' new homes and an 8K budget; checked against the old prompt on an Ask, a Plan and a planted-bug fix on a clone of a production project, outcomes matched. Since ADR-0121 Phase 7 and the backlog paragraph also say that a break your own change caused is part of the change, not an item to park. The concerns it covers:

1. **Phase gating** — labelled phases, stops between them, no mutation before Phase 6, explicit trivial fast-path.

2. **Definition of Done / match-not-improve** — 3–5 falsifiable bullets before code; Phase 7 verifies those exact bullets; adjacent improvements are surfaced ("noticed in flight, not in scope") and parked or asked — never silently landed. Real-work turns end `**Scope met:** … Anything else, or should I stop?` plus a machine sentinel for the UI chip strip.
3. **Verify-then-remediate** (v0.1.55) — mechanical failures (typecheck/tests/build) self-remediate ≤3 attempts per milestone with an early stop on identical consecutive errors; MUST NOT claim landed, weaken the DoD, or skip the failing check. Unmet DoD bullets get surface-and-offer: the gap + the one next step, then a gate. A blind retry-until-done loop was deliberately rejected.
4. **Deterministic ADR triggers** — nine MUST-write categories (foundational framework, public API shape, persistent-state schema, security boundary, default-model change, new MCP server, cross-cutting constraint, superseding, user-named) + anti-triggers + the "would future-MARVIN re-derive this in 8 weeks?" test.
5. **Async anti-fabrication** — never narrate a watcher that wasn't armed; the only honest options are block-and-wait, a background job, a wakeup, or handing back.
6. **Post-PR loop** — own the green build: run tests locally, one structured PR comment per run, fix the code under test, cap three run-fix-run cycles, name flakes honestly.
7. **Workflow health** — an injected audit block (Mode A propose / B execute / C defer) catches an in-flight project up on missed phases; a greenfield playbook adapts Phases 2–3 for empty repos.
8. **Personality** — `marvin` (dry wit, always delivers) vs `neutral`, a style layer never a refusal layer; ground-truth facts are placed first in the prompt (the highest-attention slot).

---

12. API & MCP surface

12.1 HTTP API (sidecar, `localhost:3030`)

Route groups under `sidecar/src/app/api/`: `chat` (+ `announce`, `resume`, `cancel`), `confirm`, `changes`, `files` (tree / content / raw / status / write / upload), `git` (status / diff / branch / log / push / pull / fetch), `graph`, `projects`, `sessions`, `skills` (+ `add`), `models`, `auth`, `health`, `context`, `cost`, `audit`, `backlog`, `terminal`, `honeycomb`, `worktrees`, `practice` (+ `run`, `findings`, `rules`). Details: `docs/reference/api.md`.

12.2 MCP servers

Server	Type	Tools	Gate posture
`marvin-graph`	in-process, `alwaysLoad`	16 graph tools (§3.3)	allow on `source: "sdk"` (read-only)
`marvin-memory`	in-process, deferred	`remember`, `recall`	allow (content-class enforced)
`marvin-backlog`	in-process, deferred	`backlog_add/list/update/claim/resolve/groom`	allow (content-class enforced; an over-cap body or note is refused, never cut)
`marvin-control`	in-process, `alwaysLoad`	wakeups + background jobs + worktrees	allow (bounded by rails)
`playwright`	external stdio, **opt-in**	`browser_*`	observation auto · interaction confirm · `browser_run_code_unsafe` **deny**; scouts get observation only (ADR-0045)

Browser automation defaults to the Playwright **CLI** (one-shot screenshots, scripted checks, full `playwright test`); the MCP is for stateful navigate→snapshot→assert flows, off by default because a browser subprocess per turn is heavy. MCP-vs-CLI selection is documented in the `marvin:browser` skill. A MARVIN-named tool from any source other than `sdk` confirms (ADR-0117); every other `mcp__*` tool confirms by default (ADR-0053).

---

13. Deliberate non-goals

Explicit product boundaries, each with a recorded reason (`docs/roadmap.md` "Not planned"):

- **No multi-agent implementation** — the founding bet (ADR-0001); the three read-only subagent carve-outs are not a precedent.
- **No cross-project memory** — projects are isolated workspaces (ADR-0005); a new project starts from zero, by design.
- **No hosted service / shared state** — local-first is the trust model, not a phase.
- **No cross-platform desktop (yet)** — fully-native macOS is the current commitment; x86_64 and other platforms wait for demand.
- **No auto-model heuristics** — the user routes roles to models explicitly; MARVIN doesn't guess task complexity (ADR-0002).

---

14. Building & contributing

This document is the architecture map; the practical loop lives in the repo:

- **Dev setup & build** — `README.md` ▸ *Development setup* (sidecar: pnpm + Next.js dev on `:3030`; app: `swift build` or xcodegen + Xcode in `macos/`) and ▸ *Dev loop* for running both together. `bin/marvin install-macos-app --bundled` produces the installable .app.
- **Tests** — TypeScript: `npx vitest run` from the repo root (policy, sandbox, registry, and dispatch contracts are the best-covered areas); Swift: `swift test` in `macos/`. Honest scope note: the SDK loop, the UI shells, and most API routes are opportunistically covered at best — `docs/development/testing.md` has the current map.
- **Conventions** — work lands on `development` and fast-forwards to `main`; a gitleaks pre-commit hook is installed from `scripts/hooks/pre-commit`; CI is `.github/workflows/release.yml` (tag-triggered).
- **Understanding why** — read `docs/decisions/` (the 125 ADRs) before proposing structural changes, `docs/roadmap.md` for what's in flight, and `AGENTS.md` (which `CLAUDE.md` imports) for the golden rules any agentic session in this repo is bound by. Material changes should arrive with an ADR of their own.

Compiled against the 2026-07-02 full feature inventory (51 ADRs, v0.1.55) and brought forward to v0.1.118 (125 ADRs, 2026-10-14). Corrections welcome — the ADRs are the authoritative record, and this document cites them everywhere it can.